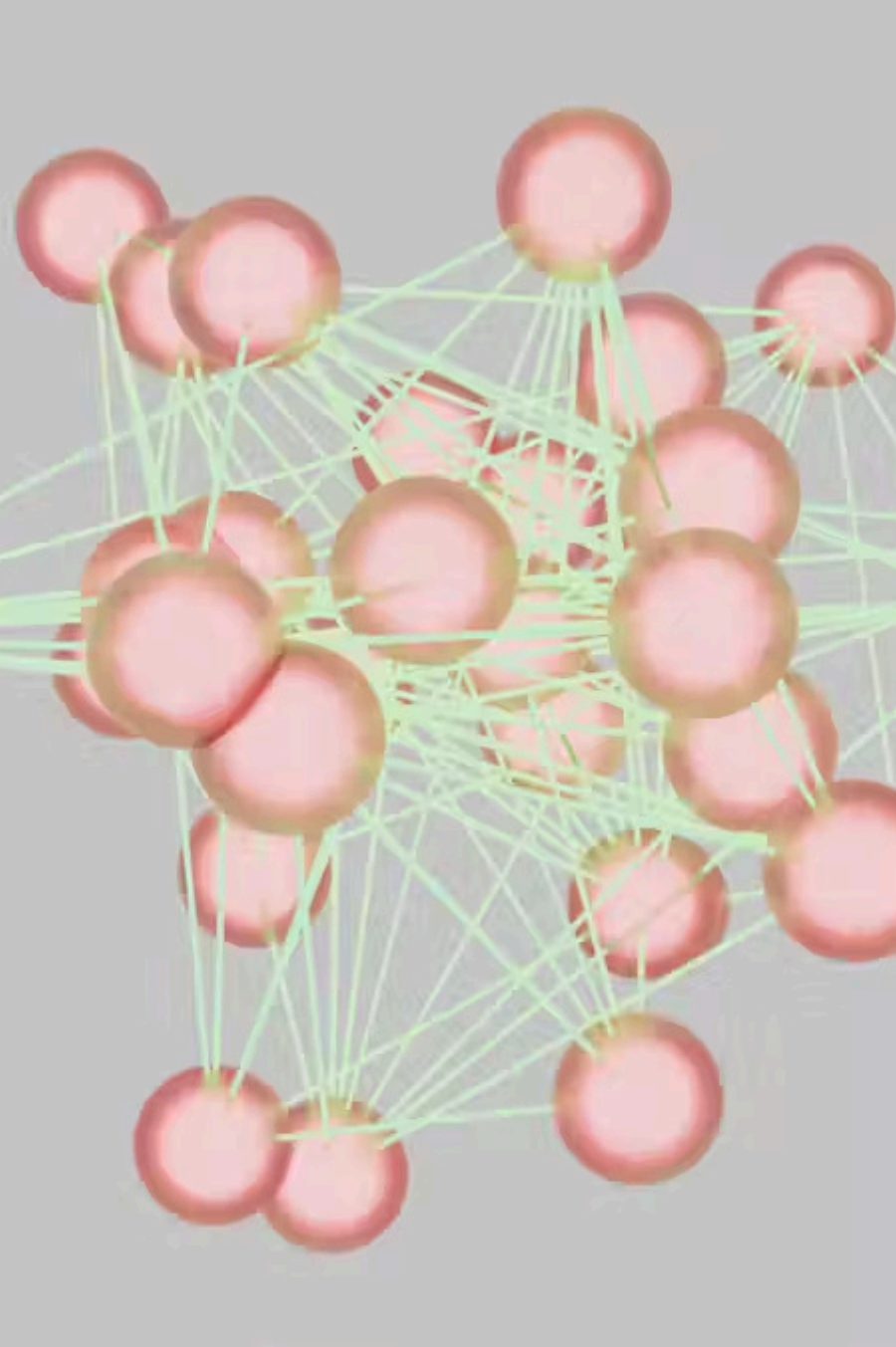




L'Injection de Dépendances avec @Autowired dans Spring

Cette présentation explore l'utilisation de l'annotation @Autowired dans Spring Framework, une méthode couramment utilisée pour l'injection de dépendances, bien que l'injection par constructeur reste la bonne pratique moderne recommandée.

Agenda

Introduction à @Autowired

Comprendre le rôle et l'importance de cette annotation dans l'écosystème Spring

Méthodes d'injection

Explorer les trois façons principales d'utiliser @Autowired dans vos applications

Exemples de code

Analyser des implémentations concrètes dans les couches Service et Controller

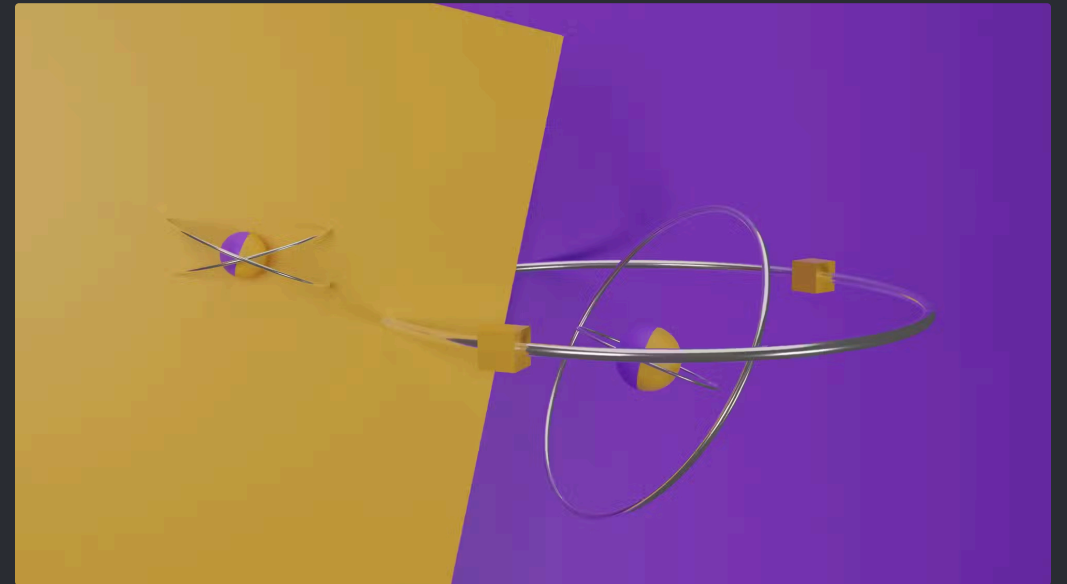
Bonnes pratiques

Recommandations pour une utilisation optimale de l'injection de dépendances

Qu'est-ce que @Autowired?

L'annotation @Autowired est un mécanisme fourni par Spring permettant d'injecter automatiquement des dépendances dans vos composants.

Elle indique à Spring qu'un constructeur, un champ, une méthode setter ou une méthode de configuration particulière doit être injecté avec l'instance appropriée d'un bean du conteneur.



Bien que largement utilisée, l'injection par constructeur (sans @Autowired explicite) est considérée comme la meilleure pratique dans les applications Spring modernes.

Les Trois Méthodes d'Injection avec @Autowired

1. Sur l'attribut

Injection directe sur le champ de classe

- ✓ Rapide et concis
- ✗ Moins propre (couplage direct)
- ✗ Difficile à tester unitairement

2. Sur le setter

Injection via une méthode setter

- ✓ Utile pour dépendances optionnelles
- ✓ Permet de changer la dépendance
- ✗ Peut créer des objets partiellement initialisés

3. Sur le constructeur

Injection via le constructeur

- ✓ Recommandée par Spring
- ✓ Dépendances obligatoires
- ✓ Facilite les tests unitaires

Exemple: Injection par Setter avec @Autowired

Service

```
@Service
public class StudentService {
    private StudentRepository repository;

    @Autowired // Spring injecte automatiquement l'implémentation
    public void setRepository(StudentRepository repository) {
        this.repository = repository;
    }

    public void saveStudent(String name) {
        repository.save(name);
    }
}
```

Dans cet exemple, Spring détecte que StudentRepositoryImpl est un bean et l'injecte automatiquement dans StudentService via la méthode setter annotée avec @Autowired.

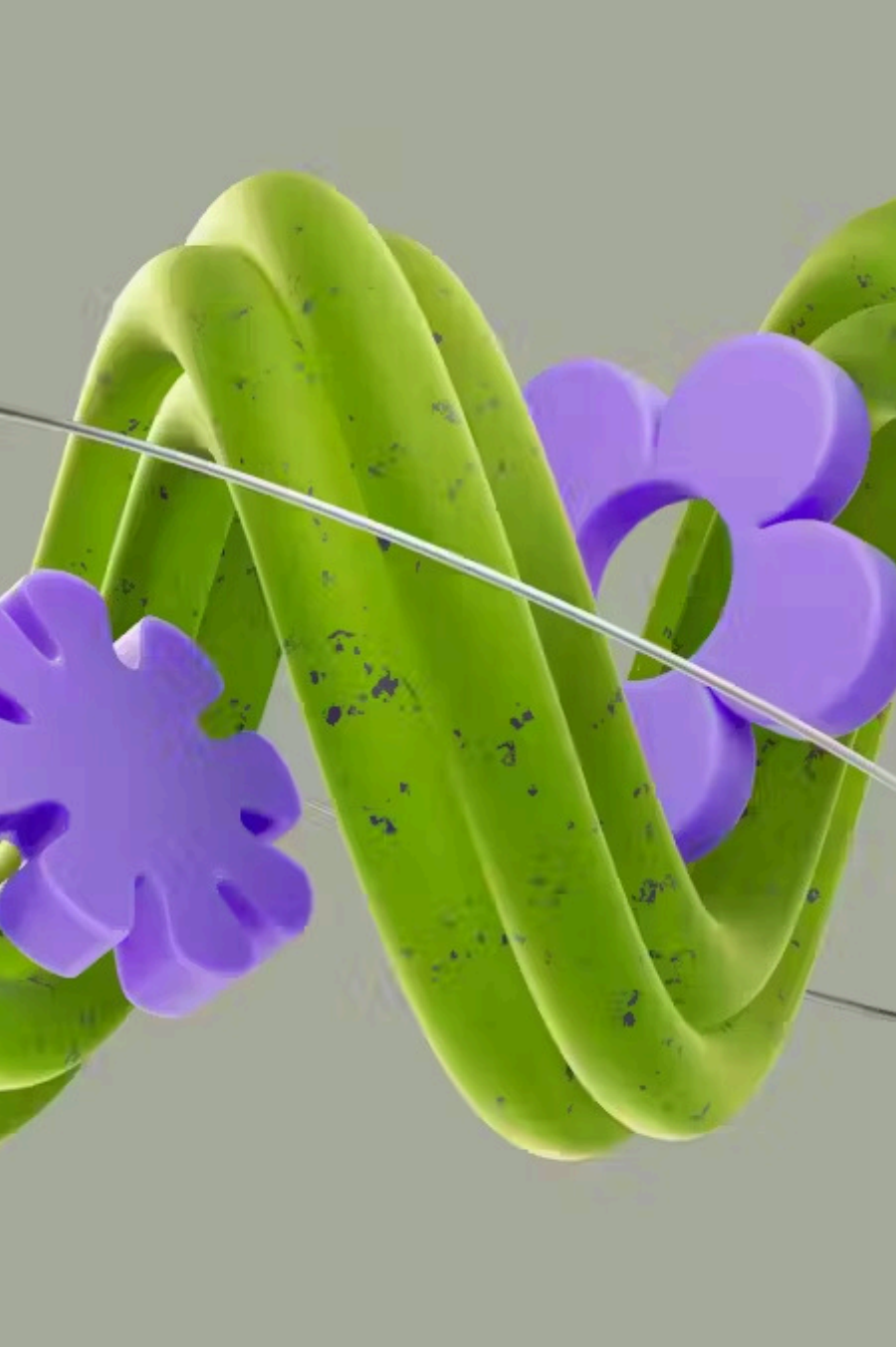
Exemple: Injection sur l'Attribut avec @Autowired

Controller

```
@RestController
@RequestMapping("/students")
public class StudentController {
    @Autowired
    private StudentService service; // Injection directe sur l'attribut

    @PostMapping("/add")
    public String addStudent(@RequestParam String name) {
        service.saveStudent(name);
        return "Étudiant ajouté avec succès";
    }
}
```

Ici, l'annotation @Autowired est placée directement sur l'attribut service. Spring injectera automatiquement une instance de StudentService dans cet attribut. Cette méthode est rapide mais moins propre car elle crée un couplage direct.



Visualisation du Flux d'Injection

Le conteneur IoC (Inversion of Control) de Spring gère le cycle de vie des beans et leurs dépendances:

1. Spring scanne les classes annotées (@Service, @Repository, @Controller, etc.)
2. Il crée des instances (beans) de ces classes
3. Il détecte les annotations @Autowired
4. Il injecte les dépendances appropriées dans chaque composant

Comparaison des Méthodes d'Injection

Critère	Sur l'attribut	Sur le setter	Sur le constructeur
Concision du code	Excellente	Moyenne	Moyenne
Testabilité	Faible	Bonne	Excellente
Dépendances optionnelles	Possible	Excellente	Difficile
Immutabilité	Non	Non	Oui
Recommandation Spring	Déconseillée	Cas spécifiques	Recommandée

Bonnes Pratiques

1 Privilégier l'injection par constructeur

Elle rend les dépendances obligatoires et facilite les tests unitaires

2 Limiter l'usage de @Autowired sur les attributs

Bien que pratique, cette approche complique les tests et crée un couplage fort

3 Utiliser les setters pour les dépendances optionnelles

Ou lorsque la dépendance peut changer pendant le cycle de vie du bean



Conclusion

Points clés à retenir

- @Autowired est un outil puissant pour l'injection de dépendances dans Spring
- Trois méthodes principales: sur l'attribut, sur le setter, sur le constructeur
- L'injection par constructeur est la méthode recommandée dans les applications modernes
- Chaque méthode a ses avantages et cas d'utilisation spécifiques



Pour aller plus loin

Explorez l'injection par constructeur sans @Autowired, qui est considérée comme la meilleure pratique dans les applications Spring modernes.

Découvrez comment Spring relie les composants à travers le conteneur IoC pour une compréhension plus approfondie de l'architecture.